Lab Assignment: Landmark Recognition Using the SIFT Algorithm

Lab Task

Develop a computer vision application that recognizes famous landmarks from query images using the Scale-Invariant Feature Transform (SIFT) algorithm. The system will maintain a database of landmark images and identify a landmark when a new image is provided, even when the query image contains changes in scale, rotation, or illumination.

Problem Statement

A tourism information center wants to develop an automated landmark recognition system. Visitors will upload photographs of famous landmarks, and the system will identify the landmark by comparing the uploaded image with a database of reference images.

Your task is to implement this application using Python and OpenCV. The system must extract SIFT keypoints and descriptors from reference images, match them against features extracted from a query image, and determine the most likely landmark based on the matching results.

Implementation Tasks

Task 1: Landmark Image Database

Create a database containing reference images of at least five famous landmarks, such as the Eiffel Tower, the Taj Mahal, the Great Wall of China, the Statue of Liberty, and the Colosseum.

Store multiple images of each landmark with different viewing angles and distances. Organize the images into separate folders according to their landmark names.

Task 2: SIFT Feature Extraction

Implement the SIFT algorithm to process the reference images.

1. Load each reference image and convert it into grayscale.
2. Detect distinctive keypoints using SIFT.
3. Extract the corresponding feature descriptors.
4. Display the reference images with their detected keypoints.
5. Store the extracted descriptors with the corresponding landmark labels.

Task 3: Query Image Processing

Develop a program that accepts an unknown landmark image as input.

1. Load the query image.
2. Convert it into grayscale.
3. Detect SIFT keypoints.
4. Extract the feature descriptors.
5. Display the query image with the detected keypoints.

Task 4: Feature Matching and Landmark Recognition

Compare the query image descriptors with the stored descriptors of each landmark.

1. Use the Brute-Force Matcher with the Euclidean distance metric.
2. Apply K-nearest-neighbor matching with (k=2).
3. Apply Lowe's ratio test to remove ambiguous matches.
4. Count the reliable matches for each landmark.
5. Rank the landmark candidates according to their matching results.
6. Display the most likely landmark and the number of reliable matches.

Use a defined matching threshold to reject unknown or incorrectly matched images. Do not classify an image solely on the basis of the largest match count.

Task 5: Landmark Localization

Extend the system to identify the location of the recognized landmark within the query image.

1. Obtain corresponding keypoints from the reference and query images.
2. Estimate a homography using RANSAC when sufficient reliable matches are available.
3. Project the four corners of the reference image onto the query image.
4. Draw a polygon around the matched landmark region.
5. Display the number of reliable matches and RANSAC inliers.

If the geometric transformation is unreliable, display an appropriate message instead of drawing an invalid polygon.

Task 6: Performance Evaluation

Test the system using query images captured under different conditions:

- Original landmark image.
- Rotated landmark image.
- Resized landmark image.
- Landmark image with reduced brightness.
- Landmark image captured from a different viewpoint.
- An image of a landmark that is not included in the database.

Record the predicted landmark, number of keypoints, number of reliable matches, recognition decision, and processing time for each test.

Expected Output

The application should display:

1. The query landmark image.
2. The detected SIFT keypoints.
3. The matched features between the query and reference images.
4. The predicted landmark name.
5. The number of reliable feature matches.
6. The localized landmark region, where geometric verification succeeds.
7. A message indicating whether the landmark was recognized or rejected.

Deliverables

Submit the complete Python source code, the organized landmark image database, screenshots of the application output, a table of experimental results, and a brief discussion of the system's performance and limitations.

The implementation should demonstrate how SIFT-based feature matching can support landmark recognition in a practical tourism information system.

1 Dataset Preparation

Collect reference images of at least five famous landmarks with different views and conditions.

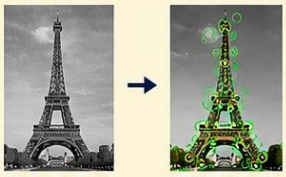


Eiffel Tower | Taj Mahal | Colosseum | Statue of Liberty | Great Wall of China

- Multiple images per landmark
- Different angles, distances and lighting
- Organize into separate folders

2 SIFT Feature Extraction

Convert images to grayscale, detect SIFT keypoints and extract descriptors for each reference image.



Grayscale Image → SIFT Keypoints

Keypoint Descriptor



0.12	0.03	0.08	0.10
0.05	0.17	0.21	0.09
0.03	0.11	0.06	0.14
0.08	0.02	0.10	0.07

128-dimensional descriptor (SIFT)

3 Query Image Processing

Input an unknown landmark image, detect keypoints and extract descriptors.



Query Image



Detected Keypoints on Query Image

4 Feature Matching

Match query descriptors with database descriptors using Brute-Force Matcher and Lowe's ratio test.



Reference Image | Query Image

- K-NN matching (k = 2)
- Lowe's ratio test
- Count reliable matches
- Rank landmark candidates

5 Landmark Recognition

Select the most likely landmark based on the number of reliable matches.

Landmark	Good Matches
Eiffel Tower	**152**
Taj Mahal	28
Colosseum	18
Statue of Liberty	12
Great Wall of China	9

Predicted Landmark:
Eiffel Tower

6 Landmark Localization

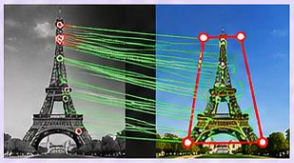
Estimate homography using RANSAC and project the reference image corners onto the query image.



Recognized Landmark with Detected Region

7 Result Output

Display the recognition result with matches and localization.



Predicted Landmark	:	**Eiffel Tower**
Good Matches	:	152
RANSAC Inliers	:	120
Processing Time	:	0.35 s
Status	:	**Recognized**

8 Testing and Evaluation

Test the system under different conditions.



Original Image | Rotated (45°) | Resized (50%) | Low Brightness | Different Viewpoint | Unknown Landmark

Applications

- Tourist information systems
- Mobile travel applications
- Augmented reality city guides
- Cultural heritage preservation
- Automatic image tagging
- Landmark search and recommendation

Expected Outcome

- Recognize famous landmarks using SIFT feature matching.
- Identify and localize the landmark in the query image.
- Achieve robustness to scale, rotation and illumination changes.
- Demonstrate the practical use of SIFT in a real-world tourism application.